```python
import gymnasium as gym
import numpy as np


def generate_episode(env, policy):
    episode = []
    state, info = env.reset()
    done = False
    while not done:
        action = np.random.choice(len(policy[state]), p=policy[state])
        next_state, reward, terminated, truncated, info = env.step(action)
        episode.append((state, reward))
        state = next_state
        done = terminated or truncated
    return episode


def first_visit_mc_prediction(env, policy, num_episodes, gamma=1.0):
    V = np.zeros(env.observation_space.n)
    returns_sum = np.zeros(env.observation_space.n)
    returns_count = np.zeros(env.observation_space.n)
    for e in range(num_episodes):
        episode = generate_episode(env, policy)
        G = 0
        for t in reversed(range(len(episode))):
            state, reward = episode[t]
            G = gamma * G + reward
            if state not in [x[0] for x in episode[:t]]:
                returns_sum[state] += G
                returns_count[state] += 1
                V[state] = returns_sum[state] / returns_count[state]
    return V


env = gym.make("FrozenLake-v1", is_slippery=False)
num_states = env.observation_space.n
num_actions = env.action_space.n
random_policy = np.ones((num_states, num_actions)) / num_actions

value_function = first_visit_mc_prediction(
    env, random_policy, num_episodes=5000, gamma=0.95
)

print("Estimated State-Value Function V(s):")
print(value_function.reshape(4, 4))
env.close()
```

```
Estimated State-Value Function V(s):
[[0.00609382 0.00678705 0.01548257 0.00497275]
 [0.00707094 0.         0.03124199 0.        ]
 [0.01681751 0.04777456 0.08417295 0.        ]
 [0.         0.11396034 0.33575285 0.        ]]
```